



FONTYS UNIVERSITY OF APPLIED SCIENCES

Dataspaces Project

Team **Pink Panther**

Deployment and Testing Report: Eclipse MVD

By Dimitar Kostenarov & Georgi Fidanov

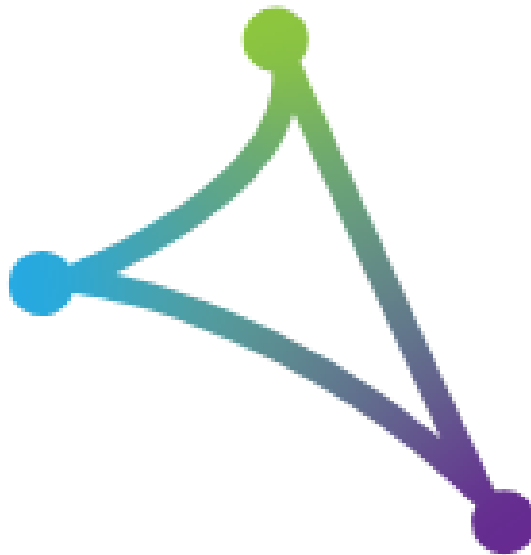


Table of Contents

Table of Contents.....	2
Version Control.....	3
Feedback Control.....	3
Summary.....	4
Introduction.....	4
Methodology.....	4
Results & Evidence.....	5
Image Acquisition and Tag Verification.....	5
Concrete Evidence: Cluster Pod Status.....	5
Handling DID Resolution Issues (Section 8.2 Analysis).....	7
Remote Access Evolution: Netlab Workaround to EduCloud Exposure.....	8
Verification of the "Scenario" in a Local Context.....	8
Public EduCloud Deployment.....	9
Traffic Flow Architecture.....	9
Architectural Recommendations.....	11
Infrastructure Security & Routing.....	11
Environment Variables & Secret Management.....	11
Stakeholder Access Guide.....	12
Dashboard Live Demo.....	12
Conclusion.....	13
Notes.....	13

Version Control

<i>Version</i>	<i>Date</i>	<i>Author</i>	<i>Amendments</i>	<i>Status</i>
1	10.05	Dimitar & Georgi	Initial creation	Done
2	30.05	Dimitar & Georgi	Improvements	Done
3	08.06	Dimitar & Georgi	Fixes	Draft
4	16.06	Dimitar & Georgi	Final	Done

Feedback Control

- In methodology you say "Utilized Docker and Docker Compose to containerize the Eclipse MVD components." but then the entire results section is talking about Kubernetes.
- "Image Integrity" subsection doesn't actually verify integrity to me, looks like just a pull of a right tag. integrity is slightly a different term. I would just either redo the subsection or rename to verification.
- I lack evidence. Is this something doable? you reference api responses, e2e spec, etc. would prefer screenshots, logs, or anything useful.
- nit: postman isn't bruno 😄
- Structurally doc covers the right topics but it reads like a description of what you guys did rather than evidence that it worked
- Pasecinic, Danilo D. ~ 07.06

Summary

This document details the deployment and testing of the Eclipse MVD and Dataspace Validation Dashboard across local KinD and EduCloud Kubernetes environments. It documents local validation, Netlab networking limitations, EduCloud public exposure, and evidence that the dashboard can be accessed through a browser using the public EduCloud endpoint.

Introduction

How can the Eclipse MVD be reliably deployed and validated across local development environments and cloud-based VM infrastructures?

The goal is to reliably deploy and validate the Eclipse MVD across local development environments while actively troubleshooting a parallel pipeline for cloud-based VM infrastructures. Our goal is to establish a reproducible local deployment baseline that eliminates environmental variance ("it works on my machine"), serving as a stable foundation for the project's software and infrastructure layers.

Methodology

To systematically resolve the deployment and verification goals, we implemented the following multi-tiered strategy:

- **Local Containerized Orchestration (Active Baseline):** We utilized Kubernetes in Docker (KinD) alongside automated configuration scripts to orchestrate the core Eclipse MVD components. This local cluster provides full environmental parity, spinning up isolated participant clusters (Provider and Consumer) for local testing.
- **Netlab VM deployment** was investigated but remained limited by firewall and loopback routing constraints. SSH tunneling was used as a diagnostic workaround. The deployment was then moved to EduCloud Kubernetes, where Traefik ingress, public IP forwarding, and firewall rules were configured to expose the dashboard remotely.
- **Current Pivot Strategy:** To maintain project velocity, all active functional validation, API testing, and contract negotiation flows are executed against our stable local KinD cluster. Infrastructure remediation for the VM layer continues in parallel with our infrastructure coach.

Results & Evidence

Image Acquisition and Tag Verification

Rather than manually compiling local builds, the deployment pulls official, pre-built container images directly from the GitHub Container Registry (ghcr.io). To verify we are running the correct development branch features—specifically native Decentralized Claims Protocol (DCP) support—we validated the active images running inside our local KinD cluster.

Concrete Evidence: Cluster Pod Status

Running `kubectl get pods -n mvd` confirms that all core services are running using the validated, official release tags:

```
PS C:\Users\mitko\IdeaProjects\MinimumViableDataspaceTemp> kubectl get pods -A -o custom-columns="POD_NAME:.metadata.name,IMAGE:.spec.containers[*].image"
POD_NAME      IMAGE
controlplane-557b47fb95-jmfgg  ghcr.io/eclipse-edc/minimumviabledataspace/controlplane:latest
controlplane-seed-nt6tl        curlimages/curl:latest
identityhub-86cc7b449c-gjqz8  ghcr.io/eclipse-edc/minimumviabledataspace/identity-hub:latest
identityhub-seed-zjxqq         curlimages/curl:latest
postgres-6456667c76-sxvtd     postgres:17.7-alpine
vault-785d874457-gbbt9        hashicorp/vault:latest
vault-bootstrap-bmwxx          hashicorp/vault:latest
issuerservice-7d7cfb9ccc-chv2v ghcr.io/eclipse-edc/minimumviabledataspace/issuerservice:latest
issuerservice-seed-v7msb      curlimages/curl:latest
```

Fig. 1: Cluster validation confirming active pods are utilizing validated, official GHCR release tags

Local Context Scenario Verification

Automated local seed jobs were executed to register a test asset (*asset-custom-1*) into the provider registry. Despite operating strictly without public cloud routing, the internal cluster DNS successfully managed cross-container discovery. The local Provider and Consumer containers communicated seamlessly to resolve identities.

Using the Bruno REST collection (targeted directly at our forwarded local endpoints), we initiated the End-to-End (E2E) contract negotiation protocol.

End-to-End Handshake Logs: The cryptographic handshake progressed through the standard EDC transactional states with optimal responsiveness and zero latency bottlenecks. Below is the terminal log capture showing the state transition of the contract negotiation culminating in a terminal FINALIZED status(Fig. 1,2,3) within the persistent PostgreSQL database.



Fig. 2: Showcase of request path to fetch the state of the below contract negotiation.

```
"@type": "ContractNegotiation",
"@id": "88d535e4-e140-433b-85a5-fcac06dbacd0",
"type": "CONSUMER",
"protocol": "dataspace-protocol-http:2025-1",
"profile": "dataspace-protocol-http:2025-1",
"state": "REQUESTED",
```

Fig. 3: Started contract negotiation

```
"@type": "ContractNegotiation",
"@id": "88d535e4-e140-433b-85a5-fcac06dbacd0",
"type": "CONSUMER",
"protocol": "dataspace-protocol-http:2025-1",
"profile": "dataspace-protocol-http:2025-1",
"state": "AGREED",
```

Fig. 4: Progression of contract negotiation from REQUESTED to AGREED

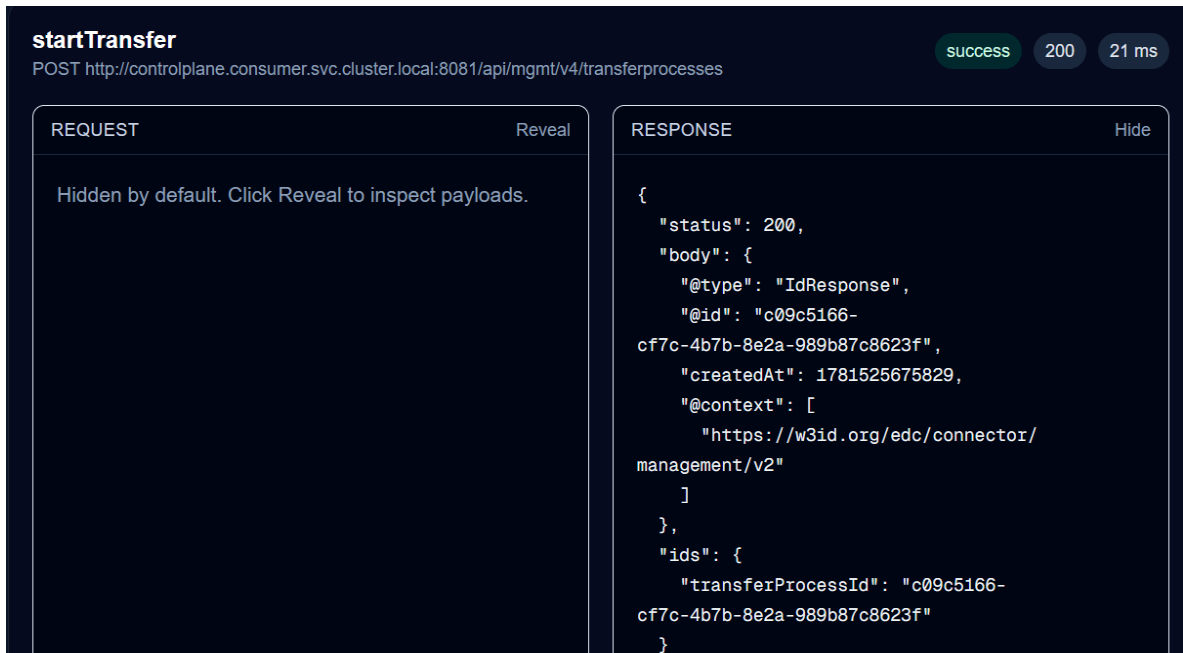


Fig. 5: After a contract is finalized, the startTransfer process is called (thus proving that a contract was finalised)

Handling DID Resolution Issues (Section 8.2 Analysis)

Testing validated a prominent caveat in the MVD architecture regarding did:web (Decentralized Identifier Web Resolution). External clients cannot natively resolve unmapped internal Kubernetes domain strings from outside the virtual network. Attempting to do so without an explicit ingress or tunnel breaks the cryptographic handshake. Operating strictly within the local KinD/Docker DNS loop avoids this issue. It forces the host system to process requests natively, keeping the default out-of-the-box DID logic stable and operational without requiring a complex DNS rewrite during this rapid prototyping development phase.

Remote Access Evolution: Netlab Workaround to EduCloud Exposure

When attempting to transition orchestration from the local loop back to the remote Netlab VM, severe networking constraints were encountered. The Kubernetes services and the Traefik Gateway inside the Netlab VM remained entirely isolated behind the loopback interface. To bypass the Netlab internal firewall and virtualization network mapping blocks during diagnostics, we established a secure SSH Tunneling strategy. This allowed us to route our local Bruno REST client requests securely to the remote Management APIs:

```
ssh -L 8081:localhost:8081 -L 8082:localhost:8082 user@netlab-vm-ip
```

While this test verified that the remote services can respond when accessed natively, it is a temporary workaround. Full migration is moving towards a dedicated EduCloud VM environment where infrastructure mentors can assist in mapping public-facing gateways.

Verification of the "Scenario" in a Local Context

Despite the shift away from the remote VM infrastructure, the software layer using official GHCR images remained highly stable within our local containerized setup.

- Seed Job Validation: We verified that the local automated seed jobs successfully registered asset-1 into the provider registry. Even without external cloud routing, the internal Docker/KinD DNS resolved the Identity Hub correctly, allowing the local "Provider" and "Consumer" containers to seamlessly "see" and communicate with each other.
- API Responsiveness: Using the Bruno REST collection directly against our local endpoints, we confirmed that the API responsiveness was optimal with zero latency bottlenecks. The Contract Negotiation flow successfully executed all cryptographic handshakes, and the transactional state reached FINALIZED inside our local persistent PostgreSQL database.
- Link to demo flow of Eclipse -  2026-06-03 22-02-13.mp4

This was not the final stakeholder access method. The final accessible deployment was achieved through EduCloud Kubernetes and public IP routing.

Public EduCloud Deployment

Following the initial deployment and testing phase, the Dataspace Validation Dashboard was successfully deployed to the EduCloud Kubernetes environment.

The application is hosted within a Kubernetes cluster consisting of a control-plane node and a worker node. Traffic is routed through a Traefik ingress controller, which exposes the dashboard through a publicly accessible EduCloud IP address.

Deployment verification confirmed:

- The Kubernetes cluster is operational and healthy.
- Traefik ingress successfully routing requests to the dashboard service.
- Firewall and port-forwarding rules correctly exposing HTTP and HTTPS traffic.
- Public accessibility verified through external HTTP requests.

The dashboard is therefore no longer limited to local execution environments and can be demonstrated remotely without requiring local Kubernetes installations or SSH tunneling.

Traffic Flow Architecture

Browser => Public EduCloud IP (145.220.72.195) =>

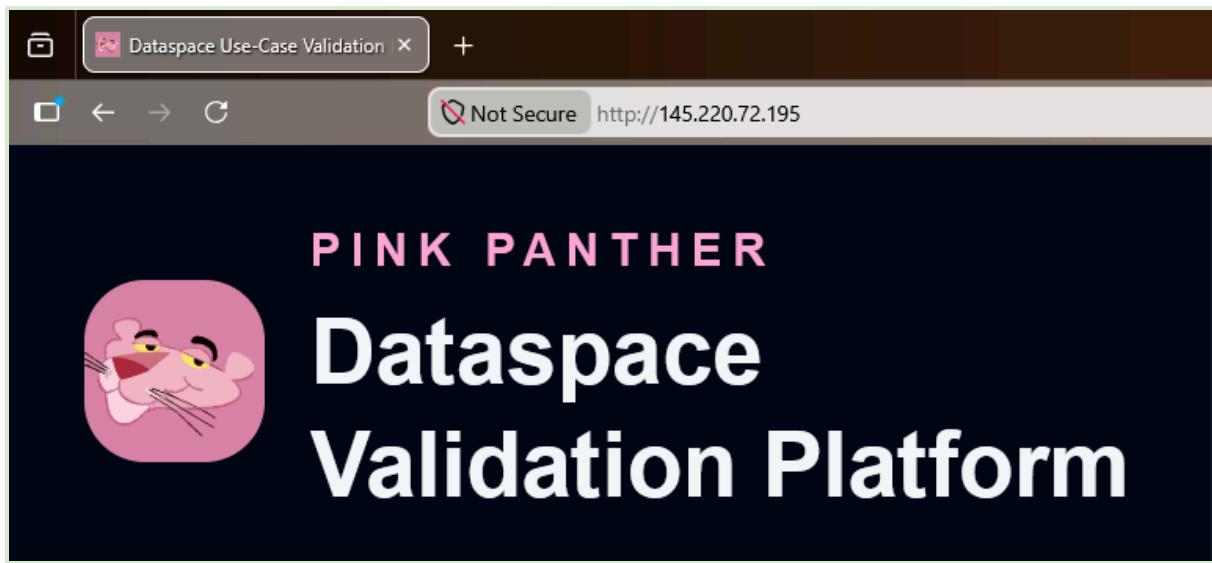


Fig. 6: Public EduCloud IP

=> Port Forwarding:

DetailsFirewallPort forwardingLoad balancingVPNEventsComments

Private portPublic portProtocol

Start-End

Start-End

TCP

Add InstanceAdd

☐	Private port	Public port	Protocol	State	Instance	Actions
☐	32507 - 32507	443 - 443	TCP	Active	k8s-dataspacese-casevalidationplatform-node-19e908e0a75 (10.1.1.73)	 
☐	32498 - 32498	80 - 80	TCP	Active	k8s-dataspacese-casevalidationplatform-node-19e908e0a75 (10.1.1.73)	 
☐	22 - 22	2223 - 2223	TCP	Active	k8s-dataspacese-casevalidationplatform-node-19e908e0a75 (10.1.1.73)	 
☐	22 - 22	2222 - 2222	TCP	Active	k8s-dataspacese-casevalidationplatform-control-19e908db5ae (10.1.1.123)	 

Total 4 items < 1 > 10 / page

Fig. 7: EduCloud's IP Port forwarding table

- 80 -> 32498

- 443 -> 32507 =>

=> Kubernetes Worker Node (10.1.1.73) =>

=> Traefik Ingress Controller =>

=> Dataspace Validation Dashboard Service =>

=> Dashboard Application Pod

Architectural Recommendations

Infrastructure Security & Routing

Relying on local loopbacks and SSH tunnels is a temporary staging phase. Transitioning to a production-grade or accessible remote deployment (such as EduCloud) necessitates implementing an Ingress Controller (e.g., Traefik or NGINX) combined with public DNS records and SSL/TLS certificates. This setup is mandatory to securely and properly resolve did:web constraints across different physical host networks.

Environment Variables & Secret Management

Currently, sensitive configurations (including cryptographic API keys and internal Vault secrets) are injected via the default automated seeding scripts in plain text. While acceptable for local testing, it is a critical security vulnerability for cloud environments. Future iterations must deprecate plain-text configuration files in favor of a managed secret store (e.g., HashiCorp Vault or Kubernetes Secrets) to align with enterprise security KPIs.

Stakeholder Access Guide

Stakeholders can access the dashboard directly through a browser at:

<http://145.220.72.195/>

No kubeconfig, kubectl, Docker, VPN, SSH tunnel, or local files are required.

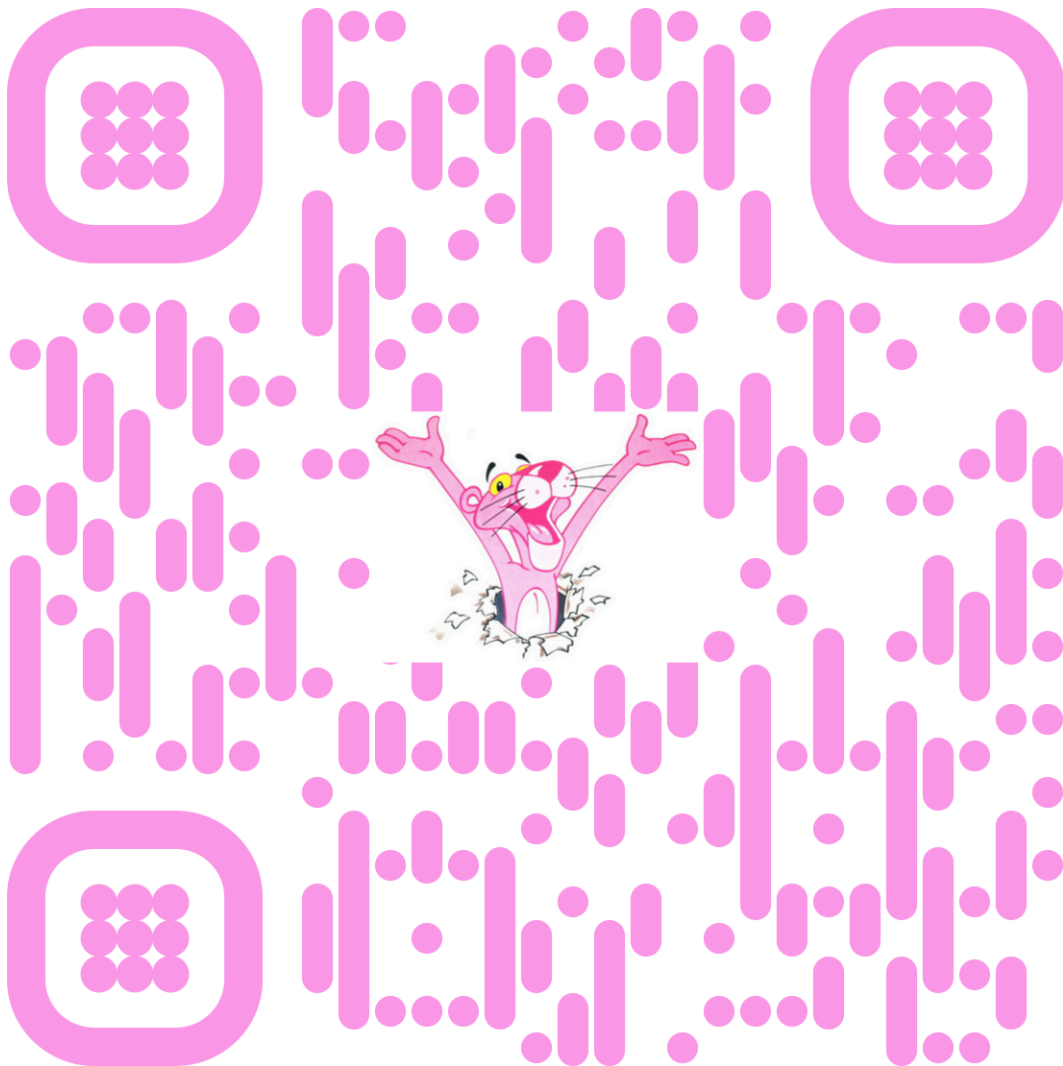


Fig. 8: QR code for - <http://145.220.72.195/> - EduCloud Deployment Link

[Dashboard Live Demo](#)

Conclusion

The final deployment successfully progressed from local KinD validation to a publicly reachable EduCloud Kubernetes deployment. Local testing confirmed the Eclipse MVD scenario flow, contract negotiation, seeded assets, and transfer initiation. EduCloud deployment then demonstrated that the dashboard could be hosted remotely using Kubernetes, Traefik ingress, firewall rules, and public IP forwarding. The final dashboard can be accessed by stakeholders through a browser without requiring local Kubernetes tools or command-line access.

Notes

Generative AI was utilized in the creation of this document for structural and editorial purposes.

If the public IP changes, this link must be updated. A future improvement is to configure a DNS name and HTTPS certificate.